# SOFTWARE REQUIREMENTS SPECIFICATION

## Hostel Management System

Web-Based Hostel Administration & Warden Management Portal

| | |
|---|---|
| **Document Type** | Software Requirements Specification (IEEE 830) |
| **Project Name** | Hostel Management System |
| **Course / Department** | IT366 – Mobile Application Development |
| **Guide Name** | Sagar Patel |
| **College Name** | CSPIT |
| **Academic Year** | 2025–2026 |
| **Date** | |

**CERTIFICATE**

This is to certify that the project entitled **"Hostel Management System"** has been successfully completed by the following student(s) under the guidance of **Sagar M. Patel**, in partial fulfillment of the requirements for the Semester 6 Mobile Application Development (IT366) course during the academic year 2025–2026.

Project Team

| Sr. No. | Student Name | Enrollment No. |
|---|---|---|
| 1 | Yash Chauhan | 23IT016 |

The work presented in this project is original and has not been submitted elsewhere. The project has been developed, tested, and deployed successfully. All the requirements specified in this Software Requirements Specification (SRS) document have been implemented and verified.

**Internal Guide** **Head of Department**

Sagar M. Patel Purvi M. Prajapati

**Date: _____** **Place: _____**

**ACKNOWLEDGMENT**

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this project.

First and foremost, I am deeply grateful to my project guide, **Prof. Sagar M. Patel**, for his invaluable guidance, constant encouragement, and constructive feedback throughout the entire duration of this project. His expertise in web application development and software engineering played a pivotal role in shaping the quality and direction of this work.

I sincerely thank **Dr. Purvi M. Prajapati**, Head of the Department of Information Technology Engineering at CSPIT, for providing the necessary academic infrastructure, resources, and a stimulating environment that made this project possible.

I am grateful to the faculty members and staff of CSPIT for their guidance in the domains of database management, web development, and system design, which formed the technical foundation of this project.

I also acknowledge the open-source community — the contributors to HTML, CSS, JavaScript, and various backend technologies — whose freely available tools and resources were instrumental in the development of this Hostel Management System.

Finally, I extend my heartfelt thanks to my family and friends whose unwavering support and encouragement were a constant source of motivation throughout this journey.

**Yash Chauhan**
Enrolment No.: 23IT016
CSPIT – IT

**ABSTRACT**

Hostel management in educational institutions is a complex administrative challenge that involves coordinating room allocations, tracking student residency, managing warden responsibilities, and maintaining accurate records — all of which have traditionally been handled through error-prone manual processes using paper registers and spreadsheets.

The **Hostel Management System** is a web-based administrative application designed to digitize and streamline the complete lifecycle of hostel operations within an educational institution. The system provides two distinct role-based access levels: **Admin** (with full system control) and **Warden** (with day-to-day operational access). The platform enables authorized users to efficiently manage room inventory, register and remove student records, monitor occupancy and availability in real time, and gain operational insights through an interactive dashboard.

Built using modern web technologies — HTML, CSS, and JavaScript for the frontend, with a Node.js / PHP backend and a MySQL / Firebase relational or NoSQL database — the system replaces fragmented manual workflows with a centralized, secure, and intuitive portal accessible from any modern web browser.

Key features include: a role-based authentication system for Admins and Wardens, a comprehensive dashboard with live occupancy statistics, a room management module for adding, editing, and deleting room records, a student management module for registering and removing student profiles, and detailed audit trails of all administrative actions.

This Software Requirements Specification (SRS) document comprehensively describes the functional and non-functional requirements, system architecture, database schema, UI/UX design principles, and future enhancement roadmap for the Hostel Management System.

**Keywords:** Hostel Management, Admin Panel, Warden Portal, Room Allocation, Student Management, Role-Based Access Control, CRUD Operations, Web Application, Dashboard, Educational Administration

# 1. INTRODUCTION

## 1.1 Background

Educational institutions in India accommodate thousands of students in hostels on a daily basis. Managing this accommodation — tracking which student lives in which room, monitoring room availability, administering fee statuses, and coordinating warden responsibilities — is an operationally intensive task. Traditionally, these activities are performed using physical registers, Excel spreadsheets, or disconnected manual systems that are prone to data inconsistency, loss, and duplication.

With the widespread adoption of internet connectivity and modern web technologies within institutions, there is a clear and immediate opportunity to replace these legacy processes with an integrated, real-time web application. A centralized Hostel Management System eliminates redundant paperwork, reduces administrative overhead, improves data accuracy, and enables faster decision-making by hostel administrators and wardens alike.

The system proposed in this document is designed specifically for educational institutions operating one or multiple hostel blocks, providing a unified digital platform for all hostel-related operations.

## 1.2 Problem Statement

The following critical challenges motivate the development of this Hostel Management System:

- **Lack of centralized data:** Student residency records, room allocations, and vacancy information are maintained in disparate formats with no single source of truth.
- **Difficulty in tracking occupancy:** Manually calculating the number of available, occupied, and reserved rooms across multiple hostel blocks is time-consuming and error-prone.
- **No role separation:** Existing manual systems do not differentiate between administrative and warden-level responsibilities, leading to data security concerns.
- **Inefficient student registration:** Registering a new student and assigning them a room requires navigating multiple disconnected steps with no automated validation.
- **No audit trail:** There is no reliable mechanism for tracking who made what change and when, making accountability difficult to enforce.
- **Absence of real-time reporting:** Generating occupancy summaries or student lists for any given time period requires manual aggregation from multiple sources.

## 1.3 Project Objectives

The Hostel Management System is designed to achieve the following primary objectives:

1. Provide a secure, role-based authentication system for **Admin** and **Warden** users with appropriately scoped access permissions.

2. Offer an interactive **Dashboard** providing a real-time snapshot of hostel occupancy, room availability, and key performance indicators.
3. Enable Admins to **add, view, update, and delete Room records** including room number, type, capacity, and current occupancy status.
4. Enable Admins and Wardens to **add, view, update, and delete Student records** including personal information, room assignment, and residency status.
5. Maintain a complete **audit log** of all critical CRUD operations performed by each user.
6. Deliver a clean, intuitive, and accessible web interface that minimizes the learning curve for non-technical administrative users.
7. Ensure data integrity and consistency through validation, database constraints, and role-based security rules.

1.4 Scope

This system targets hostel administrators (Admins) and floor-level wardens at educational institutions. The application is designed as a web-first solution accessible from any modern browser on desktop or tablet devices. The system supports a single institution with multiple hostel blocks, each containing multiple rooms. The scope of this version covers:

- Role-based login for Admin and Warden
- Full CRUD operations for Rooms and Students
- Real-time operational Dashboard
- Basic reporting and audit trail

The system does not cover online fee payment integration, biometric attendance, or student mobile application features in this version, but the architecture is designed to accommodate these extensions.

1.5 Document Conventions

| Term / Abbreviation | Definition |
|---|---|
| **SRS** | Software Requirements Specification |
| **FR** | Functional Requirement |
| **NFR** | Non-Functional Requirement |
| **Admin** | System Administrator — has full CRUD access to all modules |
| **Warden** | Hostel Warden — has limited operational access (students view, room view) |

| | |
|---|---|
| **CRUD** | Create, Read, Update, Delete — standard data operations |
| **UI / UX** | User Interface / User Experience |
| **RBAC** | Role-Based Access Control |
| **REST API** | Representational State Transfer Application Programming Interface |
| **Firebase** | Google's cloud backend platform for web/mobile applications |
| **Firestore** | Firebase NoSQL cloud database service |
| **JWT** | JSON Web Token — used for secure session management |
| **ERD** | Entity-Relationship Diagram |
| **KPI** | Key Performance Indicator — metrics displayed on the Dashboard |

## 2. OVERALL DESCRIPTION

### 2.1 Product Perspective

The Hostel Management System is a standalone web application following a three-tier client-server architecture. The web-based frontend communicates with a backend server (Node.js / PHP) which in turn interacts with the database (MySQL or Firebase Firestore). Role-based access control is enforced at both the UI and API layers to ensure data isolation between Admin and Warden users. The system does not depend on any external third-party services beyond the hosting infrastructure and authentication provider.

### 2.2 Product Features Summary

| Feature | Category | Description |
|---|---|---|
| Role-Based Login | Security | Separate login for Admin and Warden with permission scoping |
| Admin Dashboard | Analytics | Live KPIs: total rooms, occupied, available, total students |
| Room Management | Core CRUD | Add, view, edit, delete room records with occupancy status |
| Student Management | Core CRUD | Register, view, update, delete student records with room assignment |
| Warden Management | Admin-Only | Add, view, and deactivate warden accounts |
| Occupancy Tracking | Core | Real-time room availability based on student allocations |

| | | |
|---|---|---|
| Audit Trail | Compliance | Log of all CRUD actions with user ID, timestamp, and action type |
| Search and Filter | UX | Search students and rooms by multiple parameters |
| Responsive UI | UX | Functional on desktop and tablet browsers |
| Secure Session | Security | JWT-based session management with timeout |

2.3 User Classes and Characteristics

| User Class | Description | Access Level |
|---|---|---|
| **Admin** | Institution's hostel administrator. Responsible for overall system management. Configures rooms, manages wardens and students. | Full CRUD access to all modules |
| **Warden** | Floor or block-level warden. Manages day-to-day student matters within their assigned block. Moderate technical proficiency. | View and limited CRUD on Students; Read-Only on Rooms |
| **System (Automated)** | Background processes that compute aggregated dashboard statistics and update occupancy counters upon data changes. | Internal system access only |

2.4 Operating Environment

| Component | Specification |
|---|---|
| **Frontend** | HTML5, CSS3, JavaScript (ES6+) — runs in any modern browser (Chrome 90+, Firefox 88+, Edge 90+) |
| **Backend** | Node.js (Express.js) or PHP 8.x |
| **Database** | MySQL 8.0+ (relational) OR Firebase Firestore (NoSQL) |
| **Authentication** | Firebase Authentication (Email/Password) OR JWT-based custom auth |
| **Hosting** | Local server (XAMPP/WAMP) or cloud hosting (Firebase Hosting / Render / Heroku) |
| **Minimum Screen Resolution** | 1024 x 768 (desktop/tablet optimized) |
| **Network** | Internet required for cloud-based deployment; LAN-based deployment also supported |

2.5 Design and Implementation Constraints

- The Admin and Warden roles must be strictly enforced at the server level — not solely at the UI layer — to prevent privilege escalation.
- A room cannot be deleted if it currently has one or more students assigned to it; the system must enforce referential integrity.
- Student enrollment numbers must be unique across the entire system.
- All date inputs must be validated for logical consistency (e.g., check-in date must precede check-out date).
- Passwords must be stored as salted hashes (bcrypt minimum) — never in plaintext.
- The system must comply with institutional data privacy policies and not expose student personal data to unauthorized users.

2.6 Assumptions and Dependencies

- It is assumed that all users (Admin and Warden) possess basic web browser literacy.
- Room numbering and block naming conventions are assumed to be pre-defined by the institution before system setup.
- Each student is assumed to be allocated to exactly one room at any given time.
- The hosting server is assumed to be available 24/7 with a minimum uptime of 99%.
- Email addresses are assumed to be unique identifiers for all user accounts.

## 3. FUNCTIONAL REQUIREMENTS

3.1 Authentication Module

| Req. ID | Requirement Description |
|---|---|
| FR-AUTH-01 | The system shall provide a Login screen with fields for Email/Username and Password, accessible by both Admin and Warden roles. |
| FR-AUTH-02 | The system shall authenticate users against stored credentials and redirect them to their role-appropriate dashboard upon successful login. |
| FR-AUTH-03 | The system shall display a descriptive error message for invalid credentials without revealing which specific field is incorrect. |
| FR-AUTH-04 | The system shall enforce role-based access control (RBAC) — Warden users shall not be able to access Admin-exclusive modules. |
| FR-AUTH-05 | The system shall maintain an authenticated session using JWT tokens with a configurable expiry period (default: 8 hours). |

| | |
|---|---|
| FR-AUTH-06 | The system shall provide a secure logout mechanism that invalidates the current session token and clears all client-side session data. |
| FR-AUTH-07 | The system shall provide a 'Forgot Password' link that sends a password reset email to the registered address. |
| FR-AUTH-08 | The system shall lock an account after 5 consecutive failed login attempts and notify the Admin. |

3.2 Dashboard Module

| Req. ID | Requirement Description |
|---|---|
| FR-DASH-01 | The system shall display a Dashboard as the landing page immediately after successful login for both Admin and Warden users. |
| FR-DASH-02 | The Admin Dashboard shall display the following real-time KPI cards: Total Rooms, Occupied Rooms, Available Rooms, Total Students, Total Wardens. |
| FR-DASH-03 | The Warden Dashboard shall display KPIs scoped to their assigned block: Rooms in Block, Occupied, Available, Students in Block. |
| FR-DASH-04 | The Dashboard shall display a visual occupancy donut chart representing the percentage of rooms currently occupied. |
| FR-DASH-05 | The Dashboard shall display a 'Recent Admissions' list showing the last 5 students added with name, room number, and date. |
| FR-DASH-06 | The Dashboard shall display a 'Room Status Summary' table listing each block with total, occupied, and available room counts. |
| FR-DASH-07 | All Dashboard statistics shall refresh automatically without requiring a full page reload (AJAX polling or real-time listeners). |

3.3 Room Management Module

| Req. ID | Requirement Description |
|---|---|
| FR-ROOM-01 | The system shall allow Admin users to add a new room by providing: Room Number, Block Name, Room Type (Single/Double/Triple), Capacity, and Floor Number. |
| FR-ROOM-02 | The system shall validate that the Room Number is unique within its block before allowing the record to be saved. |

| | |
|---|---|
| FR-ROOM-03 | The system shall display a paginated list of all rooms with columns: Room No., Block, Type, Capacity, Occupied Count, Available Count, and Status. |
| FR-ROOM-04 | The system shall allow Admin users to edit any room record's details (type, capacity) via an inline form or modal dialog. |
| FR-ROOM-05 | The system shall allow Admin users to delete a room record, subject to the constraint that the room must have zero currently assigned students. |
| FR-ROOM-06 | The system shall display a confirmation dialog before executing any delete operation. |
| FR-ROOM-07 | The system shall allow users to search and filter rooms by Block Name, Room Type, and Availability Status. |
| FR-ROOM-08 | The system shall automatically update the room's occupancy status to 'Full' when the number of assigned students equals its capacity. |
| FR-ROOM-09 | Warden users shall have read-only access to the room list. Only Admin users may add, edit, or delete room records. |

3.4 Student Management Module

| Req. ID | Requirement Description |
|---|---|
| FR-STU-01 | The system shall allow Admin and Warden users to register a new student by providing: Full Name, Enrollment Number, Department, Year of Study, Contact Number, Parent Contact, Email, Gender, and Assigned Room. |
| FR-STU-02 | The system shall validate that the Enrollment Number is unique across all student records before saving. |
| FR-STU-03 | The system shall validate that the assigned room has sufficient available capacity before confirming the student allocation. |

| | |
|---|---|
| FR-STU-04 | The system shall automatically decrement the selected room's available capacity upon successful student registration. |
| FR-STU-05 | The system shall display a paginated list of all students with columns: Name, Enrollment No., Department, Year, Room No., Block, Status, and Actions. |
| FR-STU-06 | The system shall allow Admin and Warden users to view a detailed profile of any student record. |
| FR-STU-07 | The system shall allow Admin users to edit any student record (personal details, room assignment). |
| FR-STU-08 | The system shall allow Warden users to edit student records within their assigned block only. |
| FR-STU-09 | The system shall allow Admin users to delete a student record permanently, which shall simultaneously increment the vacated room's available count. |
| FR-STU-10 | The system shall allow users to search students by Name, Enrollment Number, Room Number, Department, or Year of Study. |
| FR-STU-11 | The system shall allow Admin users to export the complete or filtered student list as a downloadable PDF or CSV report. |

3.5 Warden Management Module (Admin-Only)

| Req. ID | Requirement Description |
|---|---|
| FR-WARD-01 | The system shall allow Admin users to add a new Warden account by providing: Full Name, Email, Password, Assigned Block, and Contact Number. |
| FR-WARD-02 | The system shall display a list of all Warden accounts with their name, assigned block, email, status (Active / Inactive), and date of joining. |
| FR-WARD-03 | The system shall allow Admin users to edit a Warden's details and reassign their block. |
| FR-WARD-04 | The system shall allow Admin users to deactivate a Warden account, preventing login without deleting their historical data. |

| | |
|---|---|
| FR-WARD-05 | The system shall prevent a Warden from accessing any modules other than Student Management (limited) and their own Dashboard. |

3.6 Reports and Audit Module

| Req. ID | Requirement Description |
|---|---|
| FR-AUD-01 | The system shall maintain an immutable audit log recording every CRUD operation: User ID, User Role, Action Type, Affected Entity, Entity ID, and Timestamp. |
| FR-AUD-02 | The Admin shall be able to view the complete audit log with search and filter by User, Action Type, Entity, and Date Range. |
| FR-AUD-03 | The system shall allow Admin users to generate and download an Occupancy Report showing room-by-room allocation details. |
| FR-AUD-04 | The system shall allow Admin users to generate a Student Residency Report filtered by Block, Department, or Year. |

---

## 4. NON-FUNCTIONAL REQUIREMENTS

4.1 Performance

- The Dashboard page shall load all KPI data and charts within **3 seconds** on a standard broadband connection.
- Any CRUD operation (add/edit/delete) shall complete and reflect on the UI within **2 seconds** of form submission.
- The student and room list pages shall support **pagination** of 20 records per page to maintain consistent load times regardless of data volume.
- The system shall support up to **100 concurrent users** without degradation in response time.

4.2 Security

- All HTTP communication between the browser and server shall be encrypted via **HTTPS/TLS**.
- User passwords shall be hashed using **bcrypt** (minimum cost factor: 10) before storage. Plaintext password storage is strictly prohibited.
- **Role-based access control (RBAC)** shall be enforced at the server/API level for every endpoint, not solely in the frontend UI.
- **JWT tokens** shall be signed with a strong secret key and set to expire after 8 hours. Expired tokens shall be rejected by the server.

- Database queries shall use **parameterized statements / prepared queries** to prevent SQL injection attacks.
- The system shall implement **CSRF protection** on all form submission endpoints.

4.3 Usability

- The core workflow (login → add student → assign room) shall be completable within **5 user interactions** by any new administrator.
- All form fields shall provide **inline real-time validation** with clear, descriptive error messages.
- **Confirmation dialogs** shall be presented before any irreversible delete operation.
- The UI shall follow **WCAG 2.1 Level AA** guidelines for colour contrast and minimum click target sizes of 44x44 pixels.
- All tables shall be sortable by the most relevant column (e.g., Student Name, Room Number, Date Added).

4.4 Reliability

- The system shall handle server or database errors gracefully with user-friendly error pages, never displaying raw stack traces.
- **Data validation** shall be enforced at both the client (JavaScript) and server layer to prevent invalid data from reaching the database.
- The system shall maintain **referential integrity** — a room cannot be deleted if students are allocated to it.
- The system shall target a minimum uptime of **99%** per month on the production hosting environment.

4.5 Scalability

- The database schema shall be designed to support a **minimum of 500 rooms and 5,000 students** without requiring structural changes.
- The application shall follow the **MVC (Model-View-Controller)** architecture pattern to facilitate horizontal scaling.
- The data model shall accommodate **multiple hostel blocks** under a single administrative instance.

4.6 Maintainability

- The codebase shall follow the **MVC design pattern** with clear separation between UI, business logic, and data access layers.
- All API endpoints shall be documented using OpenAPI / Swagger specification.

- The codebase shall achieve a minimum of **60% unit test coverage** for all backend business logic.
- Configuration values (database credentials, JWT secrets, email settings) shall be stored in environment variables — not hardcoded in source files.

## 5. SYSTEM DESIGN AND ARCHITECTURE

### 5.1 Architecture Overview

The Hostel Management System follows a classic **three-tier web architecture**:

- **Presentation Layer (Frontend):** HTML5, CSS3, and vanilla JavaScript manage all UI rendering, DOM interactions, AJAX requests, and form validations.
- **Business Logic Layer (Backend):** A Node.js (Express.js) or PHP server handles HTTP requests, enforces RBAC, orchestrates CRUD operations, computes aggregated statistics, and generates reports.
- **Data Layer (Database):** MySQL 8.0 stores structured records for Users, Rooms, Students, Wardens, and Audit Logs. Alternatively, Firebase Firestore organizes the same data as document collections.

### 5.2 Component Overview

| Component | Role | Technology |
|---|---|---|
| Web Browser Client | UI rendering, form interactions, dashboard charts | HTML5 + CSS3 + JavaScript |
| Web Server | HTTP routing, RBAC enforcement, business logic | Node.js (Express) / PHP 8.x |
| Authentication Service | Login, session management, password reset | JWT / Firebase Auth |
| Database | Persistent storage for all entity records | MySQL 8.0 / Firestore |
| Chart Engine | Occupancy bar chart, pie chart, donut chart | Chart.js (CDN) |
| Email Service | Password reset and warden invitation emails | NodeMailer / SMTP |

### 5.3 Data Flow — Add Student Workflow

1. Admin/Warden opens the **Add Student** form on the Student Management screen.
2. User fills in student details and selects an available room from the dropdown (pre-filtered to show only rooms with available capacity).

3. **Client-side validation** checks all required fields and format constraints (email, phone, enrollment number format).
4. On submission, an **HTTP POST** request is sent to /api/students/add with the form payload.
5. The **server validates** RBAC, re-validates all fields, checks enrollment number uniqueness, and confirms the room has available capacity.
6. On successful validation, the server **writes** the student record to the students table/collection.
7. The server **updates** the rooms table/collection to decrement the available count and update occupancy status.
8. An **audit log** entry is written with the user ID, action type (CREATE), entity (Student), and timestamp.
9. The server returns a **200 OK** response with the newly created student record.
10. The client updates the student list table in real time and displays a success toast notification.

5.4 Use Case Summary

| Use Case ID | Name | Primary Actor | Brief Description |
|---|---|---|---|
| UC-01 | Login | Admin / Warden | Authenticate with credentials; redirect to role-specific dashboard. |
| UC-02 | View Dashboard | Admin / Warden | View real-time KPI stats, occupancy chart, and recent admissions. |
| UC-03 | Add Room | Admin | Create a new room record with capacity and block details. |
| UC-04 | Edit Room | Admin | Update an existing room's type or capacity. |
| UC-05 | Delete Room | Admin | Remove a room record after verifying it has no assigned students. |
| UC-06 | Add Student | Admin / Warden | Register a new student and assign them to an available room. |
| UC-07 | View Students | Admin / Warden | Browse, search, and filter the complete student listing. |
| UC-08 | Edit Student | Admin / Warden | Update student personal details or room assignment. |

| | | | |
|---|---|---|---|
| UC-09 | Delete Student | Admin | Permanently remove a student record, freeing their room slot. |
| UC-10 | Manage Wardens | Admin | Add, view, edit, and deactivate warden accounts. |
| UC-11 | View Audit Log | Admin | Review a full history of all system CRUD actions. |
| UC-12 | Export Report | Admin / Warden | Download a PDF or CSV report of filtered student or room data. |
| UC-13 | Logout | Admin / Warden | Securely terminate the session and clear authentication tokens. |

## 6. DATABASE DESIGN

### 6.1 Overview

The Hostel Management System uses **MySQL 8.0** as its primary relational database (or Firebase Firestore as the NoSQL alternative). The schema consists of five core tables: users, rooms, students, wardens, and audit_logs. Foreign key constraints enforce referential integrity between related entities.

### 6.2 users Table

| Field | Data Type | Constraints | Description |
|---|---|---|---|
| user_id | INT | PK, AUTO_INCREMENT | Unique system identifier for each user |
| name | VARCHAR(100) | NOT NULL | Full display name of the user |
| email | VARCHAR(150) | NOT NULL, UNIQUE | Login email address |
| password_hash | VARCHAR(255) | NOT NULL | Bcrypt-hashed password |
| role | ENUM('admin','warden') | NOT NULL | Determines system access level |
| status | ENUM('active','inactive') | DEFAULT 'active' | Active / Deactivated account flag |

| | | | |
|---|---|---|---|
| created_at | DATETIME | DEFAULT NOW() | Account creation timestamp |
| last_login | DATETIME | NULLABLE | Timestamp of the most recent successful login |

6.3 rooms Table

| Field | Data Type | Constraints | Description |
|---|---|---|---|
| room_id | INT | PK, AUTO_INCREMENT | Unique room identifier |
| room_number | VARCHAR(20) | NOT NULL | Display label of the room (e.g., 'A-101') |
| block_name | VARCHAR(50) | NOT NULL | Hostel block name (e.g., 'Block A') |
| floor_number | INT | NOT NULL | Floor on which the room is located |
| room_type | ENUM('Single','Double','Triple') | NOT NULL | Room occupancy category |
| capacity | INT | NOT NULL | Maximum number of students the room can accommodate |
| occupied_count | INT | DEFAULT 0 | Current number of students assigned to the room |

| | | | |
|---|---|---|---|
| status | ENUM('Available','Full','Maintenance') | DEFAULT 'Available' | Current operational status |
| created_at | DATETIME | DEFAULT NOW() | Record creation timestamp |
| UNIQUE KEY | (room_number, block_name) | COMPOSITE | Ensures room numbers are unique within each block |

6.4 students Table

| Field | Data Type | Constraints | Description |
|---|---|---|---|
| student_id | INT | PK, AUTO_INCREMENT | Unique student system identifier |
| enrollment_no | VARCHAR(20) | NOT NULL, UNIQUE | Institution-issued enrollment number |
| full_name | VARCHAR(100) | NOT NULL | Student's full name |
| email | VARCHAR(150) | UNIQUE, NULLABLE | Student email address |
| contact_number | VARCHAR(15) | NOT NULL | Student's mobile contact number |
| parent_contact | VARCHAR(15) | NOT NULL | Parent/Guardian contact number |
| department | VARCHAR(100) | NOT NULL | Academic department (e.g., 'IT', 'CE') |

| | | | |
|---|---|---|---|
| year_of_study | INT | NOT NULL | Current year of study (1-4) |
| gender | ENUM('Male','Female','Other') | NOT NULL | Student gender |
| room_id | INT | FK -> rooms(room_id) | Assigned room reference |
| check_in_date | DATE | NOT NULL | Date on which student moved into the hostel |
| check_out_date | DATE | NULLABLE | Date of departure (NULL if currently residing) |
| status | ENUM('Active','Vacated') | DEFAULT 'Active' | Current residency status |
| created_at | DATETIME | DEFAULT NOW() | Record creation timestamp |
| created_by | INT | FK -> users(user_id) | Admin/Warden who registered the student |

6.5 wardens Table

| Field | Data Type | Constraints | Description |
|---|---|---|---|
| warden_id | INT | PK, AUTO_INCREMENT | Unique warden record identifier |
| user_id | INT | FK -> users(user_id), UNIQUE | Reference to the corresponding user account |
| assigned_block | VARCHAR(50) | NOT NULL | Hostel block managed by this warden |
| contact_number | VARCHAR(15) | NOT NULL | Warden's direct contact number |
| date_joined | DATE | NOT NULL | Date the warden was assigned to the hostel |

6.6 audit_logs Table

| Field | Data Type | Constraints | Description |
|---|---|---|---|
| log_id | INT | PK, AUTO_INCREMENT | Unique log entry identifier |
| user_id | INT | FK -> users(user_id) | User who performed the action |
| user_role | ENUM('admin','warden') | NOT NULL | Role of the user at the time of the action |
| action_type | ENUM('CREATE','UPDATE','DELETE','LOGIN','LOGOUT') | NOT NULL | Type of operation performed |
| entity_type | VARCHAR(50) | NOT NULL | Entity affected (e.g., 'Room', 'Student', 'Warden') |
| entity_id | INT | NULLABLE | Primary key of the affected record |

| | | | |
|---|---|---|---|
| description | TEXT | NULLABLE | Human-readable summary of the action |
| ip_address | VARCHAR(45) | NULLABLE | Client IP address at time of action |
| created_at | DATETIME | DEFAULT NOW() | Exact timestamp of the action |

## 7. USER INTERFACE DESIGN

7.1 Design Principles

- **Role-Aware Interface:** The sidebar navigation, visible modules, and action buttons adapt based on the logged-in user's role (Admin vs. Warden).
- **Dashboard-First Design:** The most actionable information (occupancy stats, recent admissions) is surfaced immediately on login to support rapid decision-making.
- **Clarity Over Cleverness:** Form layouts are linear, labels are above fields, and error messages are specific — prioritizing usability for non-technical staff.
- **Consistent Action Vocabulary:** Throughout the system, 'Add' creates, 'Edit' modifies, 'Delete' removes — always with a confirmation dialog.
- **Institutional Color Theme:** A professional palette of deep navy blue (#1A237E), clean whites (#FFFFFF), and accent greens (#43A047) conveys trustworthiness and institutional professionalism.

7.2 Screen Inventory and Descriptions

7.2.1 Login Screen

A centered login card on a gradient background displays the system logo, application title ('Hostel Management System'), and institution name. Fields: Email, Password (with show/hide toggle), and a 'Login' button. A 'Forgot Password?' link is shown below the form. Role is determined server-side post-authentication. Input validation errors are shown inline beneath each field in real time.

7.2.2 Admin Dashboard

A full-width collapsible sidebar on the left with icons and labels for: Dashboard, Rooms, Students, Wardens, Reports, Audit Log, and Settings. The main content area displays:

- **Top KPI Row:** Five stat cards — Total Rooms, Occupied, Available, Total Students, Total Wardens. Each card shows the metric value prominently with a supporting icon.
- **Occupancy Donut Chart:** A Chart.js donut chart showing occupied vs. available room ratio with a percentage label at center.
- **Block-Wise Occupancy Bar Chart:** A grouped bar chart comparing total capacity vs. current occupancy for each hostel block.
- **Recent Admissions Table:** A compact 5-row table showing the last 5 registered students: Name, Enrollment No., Room, Block, and Check-In Date.

7.2.3 Warden Dashboard

Same layout as the Admin Dashboard, but KPI cards and charts are automatically scoped to the warden's assigned block. The sidebar navigation shows only: Dashboard, Students (limited), and Profile.

7.2.4 Room Management Screen (Admin Only)

A full-width data table listing all rooms with columns: Room No., Block, Floor, Type, Capacity, Occupied, Available, Status, and Actions. An '+ Add Room' primary button at top-right opens a modal with all required fields and inline validation. Each row includes 'Edit' (pencil icon) and 'Delete' (trash icon) action buttons. Clicking Edit populates the form modal with existing data. Clicking Delete shows a confirmation dialog. A search bar and filter dropdowns (Block, Type, Status) are positioned above the table.

7.2.5 Student Management Screen

A full-width data table displaying all students with columns: Name, Enrollment No., Department, Year, Room No., Block, Status, Check-In Date, and Actions. An '+ Add Student' primary button opens the registration form including all required fields. Each student row includes 'View', 'Edit', and 'Delete' action buttons. A filter bar allows searching by name, enrollment number, room, department, and year.

7.2.6 Warden Management Screen (Admin Only)

A table listing all wardens with: Name, Assigned Block, Email, Contact, Status, and Date Joined. An '+ Add Warden' button opens a form with all required fields. Edit and Deactivate/Activate toggle actions are available per warden row.

7.2.7 Audit Log Screen (Admin Only)

A chronological, paginated table of all audit events: Timestamp, User Name, Role, Action, Entity Type, Entity ID, and Description. Filter controls allow narrowing by Date Range, User, Action Type, and Entity. Rows are color-coded by action type (green = CREATE, amber = UPDATE, red = DELETE).

7.2.8 Profile and Settings Screen

Displays the logged-in user's initials avatar (or profile photo), name, email, and role badge. An 'Edit Profile' button allows updating the display name and uploading a profile photo. A 'Change Password' section with current password, new password, and confirm password fields is provided. A 'Sign Out' button (red) is at the bottom.

## 8. FUTURE ENHANCEMENTS

8.1 Fee Management Module

Integration of a hostel fee tracking module allowing Admins to record fee payment status per student, generate fee defaulter lists, and send automated payment reminder notifications via email or SMS.

8.2 Student Mobile Application

A companion mobile application for students to view their room details, submit maintenance complaints, check hostel notices, and receive announcements from the warden — removing the need for physical notice boards.

8.3 Biometric Attendance Integration

Integration with fingerprint or RFID-based entry/exit systems to record students' hostel entry and exit times automatically, enabling curfew monitoring and safety tracking.

8.4 Maintenance Request Module

A digital complaint and maintenance tracking system where students can raise facility maintenance tickets (e.g., water issues, fan repair) and wardens can update and close the requests, providing end-to-end resolution tracking.

8.5 Room Allocation Optimization

An intelligent room allocation engine that automatically suggests the most appropriate room for a new student based on gender, department, year of study, and room preference — reducing manual allocation effort.

8.6 SMS and Email Notification System

Automated notifications to students and parents for: check-in confirmation, fee due dates, late curfew entries, and maintenance request updates.

8.7 Multi-Hostel and Multi-Campus Support

Extension of the data model and RBAC system to support multiple campuses or institution branches under a single administrative deployment, with campus-level admin roles.

8.8 Data Analytics and Reporting Dashboard

Enhanced analytics including year-over-year occupancy trends, department-wise residency distribution, average length of stay per student, and predictive vacancy forecasting for upcoming semesters.

## 9. CONCLUSION

The Hostel Management System represents a purposeful digital transformation of traditional, paper-based hostel administration into a centralized, secure, and efficient web-based platform. By providing role-based access for Admins and Wardens, the system ensures that operational responsibilities are clearly delegated while sensitive data remains appropriately protected.

The system demonstrates the practical application of full-stack web development concepts — frontend design, backend API development, relational database design, authentication and authorization, and CRUD operations — within a real-world administrative domain. The Dashboard provides immediate operational visibility, the Room and Student Management modules eliminate manual record-keeping errors, and the Audit Log introduces a level of accountability previously absent from manual hostel management processes.

From an academic perspective, the project delivers substantial learning outcomes in the areas of web application development, database design, role-based security, UI/UX design, and software requirements engineering. The architecture is intentionally extensible, with a clear roadmap for fee management, mobile applications, biometric integration, and multi-campus support that will significantly amplify the system's value in production deployment.

In conclusion, the Hostel Management System validates the principle that well-designed software — even for a bounded administrative domain — can meaningfully reduce operational friction, improve data accuracy, and enhance the day-to-day experience of both administrators and the students they serve.

## 10. REFERENCES


1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.
2. Mozilla Developer Network. (2024). *HTML5, CSS3, and JavaScript Reference Documentation*. https://developer.mozilla.org
3. Node.js Foundation. (2024). *Node.js Documentation*. https://nodejs.org/en/docs
4. Express.js Team. (2024). *Express.js Web Framework Documentation*. https://expressjs.com
5. Firebase Team. (2024). *Firebase Documentation: Firestore, Authentication*. Google LLC. https://firebase.google.com/docs
6. Chart.js Contributors. (2024). *Chart.js Documentation*. https://www.chartjs.org/docs
7. OWASP Foundation. (2023). *OWASP Top Ten Security Risks*. https://owasp.org/www-project-top-ten
8. Google Material Design 3. (2024). *Material Design Components and Guidelines*. https://m3.material.io/
9. World Wide Web Consortium (W3C). (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. https://www.w3.org/TR/WCAG21
10. IEEE Std 830-1998. (1998). *IEEE Recommended Practice for Software Requirements Specifications*. IEEE Computer Society.
11. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson Education.
12. PHP Group. (2024). *PHP Manual*. https://www.php.net/manual/en
13. MySQL AB. (2024). *MySQL 8.0 Reference Manual*. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en
14. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
15. Pressman, R. S., & Maxim, B. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.